PES VCS PROJECT REPORT

S Adithya Singh
PES2UG24CS412

Phase 1

```
=== All integration tests completed ===
adithya_singh@ideapad:~/PES2UG24CS412-PES-vcs$ ./test_objects
find .pes/objects -type f
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2

```
adithya_singh@ideapad:~/PES2UG24CS412-PES-vcs$ ./test_tree
find .pes/objects -type f | head -1 | xargs xxd | head -20
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

phase 2b

```
adithya_singh@ideapad:~/PES2UG24CS412-PES-vcs$ find .pes/objects -type f | head -1 | xargs xxd | head -20
00000000: 626c 6f62 2031 3200 6865 6c6c 6f0a 576f  blob 12.hello.Wo
00000010: 726c 640a                                rld.
```

PHASE 3

```
adithya_singh@ideapad:~/PES2UG24CS412-PES-vcs$ rm -rf .pes && ./pes init
echo "hello" > file1.txt && echo "world" > file2.txt
./pes add file1.txt file2.txt && ./pes status
cat .pes/index
Initialized empty PES repository in .pes/
Staged changes:
    staged:   file1.txt
    staged:   file2.txt

Unstaged changes:
    (nothing to show)

Untracked files:
    untracked: object.c
    untracked: tree.h
    untracked: test_sequence.sh
    untracked: .gitignore
    untracked: index.c
    untracked: README.md
    untracked: pes_debug
    untracked: pes.c
    untracked: Makefile
    untracked: test_tree
    untracked: test_objects
    untracked: commit.c
    untracked: test_tree.c
    untracked: bye.txt
    untracked: tree.c
    untracked: index.h
    untracked: test_objects.c
    untracked: commit.h
    untracked: pes.h

100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776753964 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776753964 6 file2.txt
```

PHASE 4

```
adithya_singh@ideapad:~/PES2UG24CS412-PES-vcs$ ./pes commit -m "Initial commit"
echo "World" >> file1.txt && ./pes add file1.txt && ./pes commit -m "Add world"
echo "Goodbye" > bye.txt && ./pes add bye.txt && ./pes commit -m "Add farewell"
./pes log
find .pes -type f | sort
cat .pes/refs/heads/main && cat .pes/HEAD
Committed: 7c4c78d06853... Initial commit
Committed: 104d66ca48d2... Add world
Committed: cd82a79c4500... Add farewell
commit cd82a79c45002668a7273e27169f09d138ec90f2547473d5a8c3d0e6053ac3b3
Author: Adi-2936 <adithyasingh2936@gmail.com>
Date:   1776754045

    Add farewell

commit 104d66ca48d239c321a69e42714782a9ae7ad9a755cf0ea2384c6e02c1e1a8aa
Author: Adi-2936 <adithyasingh2936@gmail.com>
Date:   1776754045

    Add world

commit 7c4c78d068535f71c6af0511362750df35e8741a9fd0c2a6cd583f7d9460d0b7
Author: Adi-2936 <adithyasingh2936@gmail.com>
Date:   1776754045

    Initial commit

.pes/HEAD
.pes/index
.pes/objects/10/4d66ca48d239c321a69e42714782a9ae7ad9a755cf0ea2384c6e02c1e1a8aa
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/34/fb83796d921969e6ba2a0c30dac825ea371b12576ff5709ffb9064f2112da4
.pes/objects/7c/4c78d068535f71c6af0511362750df35e8741a9fd0c2a6cd583f7d9460d0b7
.pes/objects/b4/e76aee38854839557e5a3132e346adbf8288ec2ba878f2cd915d5c2326af00
.pes/objects/cd/82a79c45002668a7273e27169f09d138ec90f2547473d5a8c3d0e6053ac3b3
.pes/objects/ce/ed54744634f312e9a7da9be821ec651ce533d1629bf9dd90608d325e896893
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f5/902ee73b6aa25436f687067fc4cc0cb481cf416740d3c3c3ed6d9c4de7b6c2
.pes/refs/heads/main
cd82a79c45002668a7273e27169f09d138ec90f2547473d5a8c3d0e6053ac3b3
ref: refs/heads/main
```

FINAL TEST

```
adithya_singh@ideapad:~/PES2UG24CS412-PES-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
    staged:   file.txt
    staged:   hello.txt

Unstaged changes:
    (nothing to show)

Untracked files:
    (nothing to show)


--- First Commit ---
Committed: dc2401ab61e2... Initial commit

Log after first commit:
commit dc2401ab61e25fffae2b09c7f0cb21cb2c259bc7fff9c2d18aecba164e7a1112
Author: Adi-2936 <adithyasingh2936@gmail.com>
Date:   1776754100

    Initial commit


--- Second Commit ---
Committed: 16e4660451c6... Update file.txt

--- Third Commit ---
Committed: a370d1c1cb80... Add farewell

--- Full History ---
commit a370d1c1cb80bc1f80fbb1c3370953253568a24def4900f0b54349e75e560aa6
```

```
--- Full History ---
commit a370d1c1cb80bc1f80fbb1c3370953253568a24def4900f0b54349e75e560aa6
Author: Adi-2936 <adithyasingh2936@gmail.com>
Date:   1776754100

    Add farewell

commit 16e4660451c6da466a22d294d9e9966483cef02610e61c55c1d0eb928921cd25
Author: Adi-2936 <adithyasingh2936@gmail.com>
Date:   1776754100

    Update file.txt

commit dc2401ab61e25fffae2b09c7f0cb21cb2c259bc7fff9c2d18aecba164e7a1112
Author: Adi-2936 <adithyasingh2936@gmail.com>
Date:   1776754100

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
a370d1c1cb80bc1f80fbb1c3370953253568a24def4900f0b54349e75e560aa6

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/16/e4660451c6da466a22d294d9e9966483cef02610e61c55c1d0eb928921cd25
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/a3/70d1c1cb80bc1f80fbb1c3370953253568a24def4900f0b54349e75e560aa6
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/dc/2401ab61e25fffae2b09c7f0cb21cb2c259bc7fff9c2d18aecba164e7a1112

=== All integration tests completed ===
```

Phase 5 & 6: Analysis Questions

Q5.1 — How would you implement pes checkout <branch>?

To implement checkout, the following steps must occur in sequence.

First, read the target branch reference from .pes/refs/heads/<branch> to obtain the commit hash. Then, read the corresponding commit object to extract its tree hash. Using this tree hash, walk the tree recursively to reconstruct the complete snapshot of the repository, which includes all file paths and their associated blob hashes.

Next, update the working directory. For each file in the target tree, read its blob from the object store and write it to disk. At the same time, any files that exist in the current working tree but are not present in the target tree must be deleted.

After updating the working directory, update the HEAD file by writing “ref: refs/heads/<branch>” so that it points to the new branch. Finally, update the index so that it matches the target tree exactly.

What makes this complex is handling uncommitted changes. If the user has modified files that differ between branches, blindly overwriting them would cause data loss. Therefore, conflicts must be detected before proceeding. Additionally, updating the working directory requires careful ordering and should be done in a way that avoids leaving the repository in a corrupted state if interrupted.

Q5.2 — How to detect a "dirty working directory" conflict during checkout?

To detect whether a file would be lost during a branch switch, first load the current index, which represents the staged state of files.

Then, for each file in the target branch's tree, check whether it differs from the current branch's version. If a file differs and is also modified in the working directory, the checkout must be refused.

Modification can be detected without re-hashing by comparing metadata stored in the index with the actual file on disk using stat(). If the modification time or size differs from what is stored in the index, the file is considered dirty. In such cases, checkout should abort with an appropriate error message indicating that local changes would be overwritten.

Q5.3 — What is "Detached HEAD" and how to recover?

A detached HEAD state occurs when the HEAD file contains a direct commit hash instead of a reference to a branch. This typically happens when checking out a specific commit rather than a branch name.

In this state, new commits can still be created, but they are not associated with any branch. As a result, these commits can become unreachable if the user switches to another branch, since no reference points to them.

Recovery is possible if the commit hash is known. A new branch can be created by writing the commit hash into a new file under .pes/refs/heads/ and updating HEAD to point to this new branch. This makes the commits reachable again and prevents them from being lost during garbage collection.

Q6.1 — Garbage Collection Algorithm

The goal of garbage collection is to identify and remove all objects that are not reachable from any branch reference.

The process begins by initializing an empty set of reachable objects. For each branch in .pes/refs/heads/, read the commit hash and traverse the commit history. For every commit encountered, add its hash to the reachable set, then follow its tree hash and recursively traverse all trees and blobs. Continue following parent pointers until reaching the initial commit.

After marking all reachable objects, scan the entire object store in .pes/objects/. Any object not present in the reachable set can be safely deleted.

An efficient data structure such as a hash set should be used to allow constant-time membership checks.

For a repository with around 100,000 commits and 50 branches, assuming each commit references multiple trees and blobs, the total number of objects processed could reach several hundred thousand to over a million. Garbage collection must therefore be efficient in both traversal and lookup operations.

Q6.2 — GC Race Condition with Concurrent Commits

A race condition can occur if garbage collection runs concurrently with a commit operation.

For example, a commit process may write new objects to the object store but has not yet updated the branch reference. If garbage collection runs at this moment, it may consider these new objects unreachable and delete them. This leads to repository corruption because the commit being constructed depends on those objects.

To prevent this, systems like Git use multiple safeguards. Newly created objects are not immediately eligible for deletion, typically using a time-based threshold. Additionally, locking mechanisms are used during reference updates to prevent concurrent modification issues. By ensuring that objects are written before references are updated and by delaying deletion of recent objects, the system avoids deleting data that is still in use.